

Instrucciones Rápidas — FCH AutoLab PartsBot

Instalación en 1 solo paso (recomendado)

Ya no tienes que ejecutar todo a mano. Hay un script que hace **TODO** (entorno virtual, dependencias, base de datos, catálogo, marcas chinas con IA y piezas):

Windows:

```
setup.bat
```

Linux / macOS:

```
bash setup.sh
```

El script te pedirá completar el `.env` (credenciales CassChoice + `GEMINI_API_KEY`) y luego hace el resto solo. Al terminar puedes arrancar el servidor con `iniciar_servidor.bat` (Windows) o `python -m uvicorn main:app --reload`.

 Necesitas una `GEMINI_API_KEY` gratuita (<https://aistudio.google.com/app/apikey>) para que se completen las marcas chinas con IA.

Comandos de IA por separado (si prefieres control manual)

```
# Poblar TODAS las marcas chinas (modelos + años) en el catálogo de vehículos
python sincronizador.py --completar-marcas

# Completar las compatibilidades (fitment) de piezas ya cargadas sin modelo
python sincronizador.py --completar-fitment
```

Cambios Implementados (Última actualización)

1. Paleta de Marca FCH AutoLab

- **Verde lima:** `#D1F645` (botones de acción, badge FCH)
- **Gris:** `#D6D8DC` (bordes, superficies secundarias)
- **Negro:** `#111418` (header, texto principal)

 El texto sobre el verde lima usa negro para máximo contraste y legibilidad.

2. Login Automático a CassChoice

Ya no necesitas copiar el SID y el token a mano del navegador.

Edita tu `.env` y pon:

```
CASS_USUARIO=fchgarageusa@gmail.com
CASS_PASSWORD=Aa123#456
```

El sistema obtendrá automáticamente:

- `sid` (cookie de sesión)
- `X-Frappe-CSRF-Token`

cuando ejecutes el sincronizador o inicies el servidor.



3. Bugs Corregidos en el Sincronizador

Bug #1: `vehicle_details` como texto

La API de CassChoice envía compatibilidades como strings:

```
"CHANGAN Alsvin 2018~2024"
```

No como diccionarios. Se agregó `_parse_vehicle_detail_str()` que:

- Extrae la marca: `CHANGAN`
- Extrae el modelo: `Alsvin`
- Extrae el rango de años: `2018` → `2024`

Bug #2: Códigos OEM/Aftermarket sin clasificar

Los códigos adicionales (OEM, aftermarket) venían en `replace_parts_numbers` pero no se leían. Ahora se clasifican por `brand_code`:

- `CHANGAN_OEM` → código OEM
- `CHANGAN_AM` → código aftermarket
- `CHANGAN` → código OE/original

Bug #3: Descripción vacía

Algunos productos vienen con `product_title=""`. Ahora se busca el primer producto con título/categoría no vacío.



4. Importador Offline (`importar_merchant.py`)

Puebla la base de datos con datos reales desde un archivo JSON (como `merchant.txt`) **sin necesidad de credenciales en vivo**.

Uso:

```
python importar_merchant.py merchant.txt
```

Resultado:

- ☒ 2.798 vehículos importados
- ☒ Piezas con precio FOB + **6% de margen** automático

- ☒ Compatibilidades agrupadas por rango de años
- ☒ Códigos OE/OEM/aftermarket clasificados

Prueba Rápida

1. Inicializar la base de datos

```
python init_db.py
```

2. Importar datos reales (modo demo)

```
python importar_merchant.py merchant.txt
```

3. Levantar el servidor

```
uvicorn main:app --reload
```

4. Probar búsqueda por marca

```
curl "http://localhost:8000/consultar?marca=CHANGAN"
```

Respuesta esperada:

```
{
  "total": 1,
  "resultados": [
    {
      "numero_oem": "3705010-H01",
      "descripcion": "Ignition coil",
      "codigo_oe": "EA0120200",
      "codigo_oem": "EA012-0200",
      "codigo_aftermarket": "EA012-0200",
      "precio_venta": 17.5521,
      "compatibilidades": [
        {
          "marca": "CHANGAN",
          "modelo": "Alsvin",
          "desde": 2018,
          "hasta": 2024,
          "etiqueta": "CHANGAN Alsvin (2018-2024)"
        }
      ]
    }
  ]
}
```

Estado de la Base de Datos

Después de ejecutar el importador con `merchant.txt` :

Tabla	Registros
<code>catalogo_vehiculos</code>	2.798
<code>autopartes</code>	1
<code>compatibilidades</code>	1
<code>precios_casschoice</code>	2
<code>traducciones_partes</code>	55

Validación de Login Automático

El login automático con tus credenciales fue probado exitosamente:

✓ Login exitoso!
 SID: db025816145d1a3cbff9...
 Token: (obtenido de página /store/)
 Total nodos descargados: 129 marcas raíz

Archivos Clave Creados/Modificados

Archivo	Descripción
<code>index.html</code>	Frontend con paleta FCH AutoLab
<code>cass_client.py</code>	Método <code>login()</code> para obtener SID/token automáticamente
<code>config.py</code>	Nuevas variables <code>CASS_USUARIO</code> y <code>CASS_PASSWORD</code>
<code>sincronizador.py</code>	Fixes de parseo y clasificación de códigos
<code>importar_merchant.py</code>	★ Importador offline de datos reales
<code>.env.example</code>	Actualizado con instrucciones de login automático

Siguiente Paso: Sincronización Completa en Vivo

Para sincronizar el catálogo completo desde CassChoice (no solo la muestra del merchant.txt):

```
# Sincronizar solo vehículos
python sincronizador.py --solo-vehiculos

# Sincronizar piezas específicas
python sincronizador.py --piezas F4J16-3705110AB 3705010-H01

# 0 desde un archivo
python sincronizador.py --archivo-piezas seed_parts.txt
```

El sistema usará automáticamente las credenciales de `.env` para hacer login.

Marcas chinas: compatibilidades con IA

El problema: CassChoice **no entrega modelos ni años** (`vehicle_details` viene vacío) para la mayoría de las marcas chinas (CHERY, JETOUR, CHANGAN, GEELY, HAVAL, MG, BYD...). Por eso, al sincronizar una pieza china como `F4J16-3705110AB`, salían **0 compatibilidades**. No es un error del código: es una limitación del proveedor.

La solución: el sistema usa **IA** para inferir el fitment (modelo + rango de años) a partir del número de parte, la marca y la descripción. Las compatibilidades inferidas se guardan con `origen="ia"` y se muestran en el catálogo con una etiqueta **IA** (badge verde) y la nota “· estimadas por IA” para que se distingan de las oficiales.

Cómo activarla en tu máquina local

1. Consigue una API key gratuita de Google Gemini en <https://aistudio.google.com/app/apikey>
2. Agrégala a tu archivo `.env` :
`GEMINI_API_KEY=tu_api_key_aqui`
3. ¡Listo! A partir de ahí, cada vez que sincronices una pieza china sin fitment, la IA lo completa automáticamente.

Uso

- **Automático al sincronizar:** si una pieza queda con 0 compatibilidades, la IA las infiere sola.

```
bash
python sincronizador.py --piezas F4J16-3705110AB
# (usa --sin-ia para desactivar la inferencia por IA)
```

- **Completar piezas ya cargadas** (endpoint autenticado):

```
POST /ia/completar_fitment -> procesa las piezas SIN compatibilidades
POST /ia/completar_fitment {"autoparte_id": 1} -> una pieza concreta
```

Si no configuras `GEMINI_API_KEY`, el sistema sigue funcionando normalmente; simplemente no rellena las compatibilidades de las marcas chinas (las deja vacías en lugar de fallar).

Prueba real: la pieza CHERY `F4J16-3705110AB` pasó de **0** → **5-8 compatibilidades** (Arrizo 5/6, Tiggo 4/7/8, y modelos JETOUR que comparten plataforma).



Notas Importantes

1. **Localhost del Agente:** Este código corre en la VM del agente Abacus AI. Para ejecutarlo en tu máquina local, descarga los archivos usando el botón **“Files”** en la esquina superior derecha.
 2. **Margen del 6%:** Se aplica automáticamente sobre el precio FOB. Si el precio FOB es `16.5586 USD` , el precio de venta será `17.5521 USD` .
 3. **Seguridad:** El archivo `.env` con tus credenciales está en `.gitignore` y **no se subió a GitHub**.
 4. **PR Mergeado:** El Pull Request #2 fue mergeado exitosamente a `main` y todos los cambios ya están en el repositorio.
-

Repositorio actualizado: <https://github.com/fjcht/partsbot>